

Modelling the Relationship between Particle count and Localization Error in Monte Carlo Localization to Determine an Optimal Number of Particles Across Different Environments

March, 26, 2026

1 Introduction

In modern robotics one of the most fundamental components to take into consideration is error in position estimation. Since these robots rely on various different sensors to determine their location, there is always at least some uncertainty in their calculated position. Within a confined space, and using distance sensors, a common method to address this uncertainty is using Monte Carlo Localization (MCL), which estimates a robot's position using a set amount of randomly distributed particles and assigning weights to each particle that better match the sensor readings. This paper investigates the how the number of particles N relates to the distance error to determine the optimal number of particles under different environmental conditions.

As a member of a VEX robotics team, I have personally experienced challenges of accurately determining a robot's position during autonomous periods within a confined field, which influenced my interest in the probabilistic localization algorithm and its effectiveness in reducing error.

2.1 Physical Measurements

The robot obtains data about its surroundings through optical distance sensors. These sensors measure the distance in which an object is found by emitting light and detecting the point in which reflection occurs. In this application, there are 2 optical sensors used: one measuring directly in front of the robot and one measuring towards the right. While other orientations such as pointing left or backwards could be used, for simplification, forward and right will be used in this investigation without affecting the results. Additionally, the robot has another sensor, the inertial measurement unit (IMU) that uses a gyroscope to provide an estimate of its orientation θ .

Initially, the robot does not know where it is located in the confined space. Let X be its global position inside the field represented as a vector

$$X = (x, y, \theta)$$

The x value represents its horizontal position relative to the left of the field, and the y value represents the vertical position relative to the top of the field. The theta value will be with reference to a flat side of the square perimeter. For consistency across calculations, all distances in this investigation will be measured in meters.

Next, the sensor readings will be represented as a vector Z

$$Z = (d_{front}, d_{right})$$

where d_{front} represents the distance in which the optical sensor hits an object in front of it, and d_{right} represents the distance in which the optical sensor hits an object on the right of it.

The orientation θ is not included in the measurement vector and is only accounted for as a reference for generating particles. It is assumed that it is measured by the gyroscope. Given any

value of X , a corresponding Z can be determined as the sensor readings are functions of the robot's position and orientation. Therefore, the sensor readings can be expressed as a function of the robot's position and orientation where $Z = f(X)$. Different positions of X produce a corresponding sensor output Z

2.2 Limitations of the Model

The first limitation in this model is the non-unique solutions. For example, in the case of a square field with no obstructions, it is possible that multiple different values of X return the same sensor readings Z . This happens when both the front sensor and the right sensor detect the same wall instead of hitting different walls. Therefore, a single point cannot be uniquely determined as multiple particles will share the same weight, even with different values of X . While increasing the number of sensors (to pointing on each side) reduces or could possibly eliminate ambiguity in the system, for the purpose of this investigation and to obtain a clearer analysis of the relationship between the number of particles N and localization accuracy I use a limited sensor configuration.

The second limitation is with movement. In this investigation, I am running a static simulation where the robot has no prior knowledge of its previous positions, and each trial's first iteration is fully randomized. Although movement heavily reduces the possibilities of non-unique solutions being present, for keeping consistent data and maintaining the core relationship between N and localization error, it is not accounted for.

3.1 Probabilistic Mathematical Distributions

Mathematically, the goal is defined as the probability of the robot being at any given coordinates with a theta value given the conditions of Z

$$P(x, y, \theta | Z)$$

To determine the robot's position, the algorithm must first calculate the difference between the real and tested sensor values for calculating the determined weight of the particle. This value is then squared so that the magnitude of the difference is taken, and to furthermore penalize large errors. This is done through

$$e_i = (Z_{tested} - Z_{actual})^2$$

$$e_i = (d_{f, tested} - d_{f, actual})^2 + (d_{r, tested} - d_{r, actual})^2$$

where Z_{actual} is the physically read sensor value and Z_{tested} is the tested sensor value.

Next, we implement the Gaussian distribution formula to assign a higher weight to closer particles and lower weight to further particles. Other types of distribution curves are less suitable for this problem. Linear/absolute value graphs do not properly filter out large errors and do not decrease fast enough. Although negative parabolic values are symmetrical and assign high weights to values close to the real sensor readings, these graphs may produce negative values which are not applicable to probability distributions. The Gaussian distribution is most suitable because it provides a smooth curve which decreases weight exponentially as the error increases, and furthermore reflects realistic sensor uncertainty. The formula used is

$$w_i = e^{-\frac{e_i}{2\sigma^2}}$$

where w_i is the weight of the particle. The constant in the normal distribution bell curve is removed because weight is assigned relatively, not absolutely. The parameter σ controls how quickly the weight decreases in relation to an increasing error. A smaller value of σ results in a stricter filtering process, while a larger value allows for more variation. Therefore, the particles which are closer to the real sensor data are assigned a higher weight and have a higher likelihood.

$$w_i \propto P(Z | X_i)$$

where Z represents the observed sensor data and X_i represents the position and orientation of the i -th particle.

3.2 Particle Filtering Process (Normalization and Resampling)

Furthermore, probability is determined through examining the total weights and normalizing them, turning these weights into probabilities which all add up to 1. The normalization is determined through

$$w'_i = \frac{w_i}{\sum_{j=1}^N w_j}$$

After each particle is calculated for, w'_i represents the probability of the robot being at each particle given the sensor data. To implement the usage of these probabilities, a random selection is made based on the probabilities. Each particle can be chosen multiple times, or none at all depending on the probability of it being chosen. This makes the program focus on the most

likely positions while filtering out the particles that have large errors relative to the read sensor readings. This process approximates the probability of $P(X | Z)$.

3.3 Final Position Estimation and Error Calculations

After resampling the data, the particles are now mostly concentrated around the likely positions of the robot. Finally, the final estimated position is found by averaging the resampled data.

Weight has already been considered for in the calculation because the simulation can choose each particle more than once.

$$\bar{x} = \frac{1}{N} \sum_{j=1}^N x_j$$

$$\bar{y} = \frac{1}{N} \sum_{j=1}^N y_j$$

Although each particle contains an orientation of θ , this investigation is about positional accuracy. Therefore, the calculation for the error only requires the horizontal and vertical components in its final estimate, while the angle was used in the weighting process. Therefore, I can model the relationship between N and the localization error measured with the distance formula.

$$E = \sqrt{(x_{true} - \bar{x})^2 + (y_{true} - \bar{y})^2}$$

4 Experimental Methodology

Using the C++ coding language, I simulated the robot's behaviour in an identical setting of a VEX robotics field using the same perimeter size (3.65m x 3.65m) and the maximum robot size of 0.457m x 0.457m. There will be sensors attached to the middle of the front of the robot, as well as the middle of the right of the robot for distance sensors.

For the initial generation of particles, I will be using random uniform spatial distribution given that the robot does not know anything at all except for the theta value. So the horizontal and vertical components of the generation of particles will be represented as

$$x_i \sim Uniform(0, 3.65m)$$

$$y_i \sim Uniform(0, 3.65m)$$

and the theta value will be determined using a Gaussian distribution, meaning that the angle is centered around theta and its spread is controlled by a constant standard deviation

$$\theta_i \sim N(\theta, \sigma^2)$$

The constant standard deviation in my program is $8^\circ = 0.139626 \text{ rad}$. This is because this value produces a reasonable variance in particle orientation while still remaining close to the true orientation. The results will display the number of particles in relation to the mean localization error. In addition, to incorporate different environmental conditions, there will be three configurations to examine how obstacles affect the results. The first environment will be the plain square field, the second environment will have a square in the middle of the field with the same dimensions as the robot, and the third will have 4 squares placed in each corner of the field with a total area equal to the area of the robot.

Furthermore, each localization error will be the average error among 1000 trials, where the robot's position is randomized, and each trial is treated independently.

5 Results

N	No Obstacles (m)	Center Pillar (m)	4 Corners (m)
50	0.4749	0.7787	0.4354
100	0.3783	0.6702	0.3317
150	0.3255	0.6352	0.2941
200	0.3048	0.6335	0.2895
300	0.2828	0.6087	0.2592
400	0.2573	0.5819	0.2336
500	0.266	0.5892	0.2183
750	0.2487	0.5219	0.1879
1000	0.2293	0.52	0.2041

Figure 1: Mean Localization Error across 1000 Trials in Relation to N Number of Particles Under Different Environments.

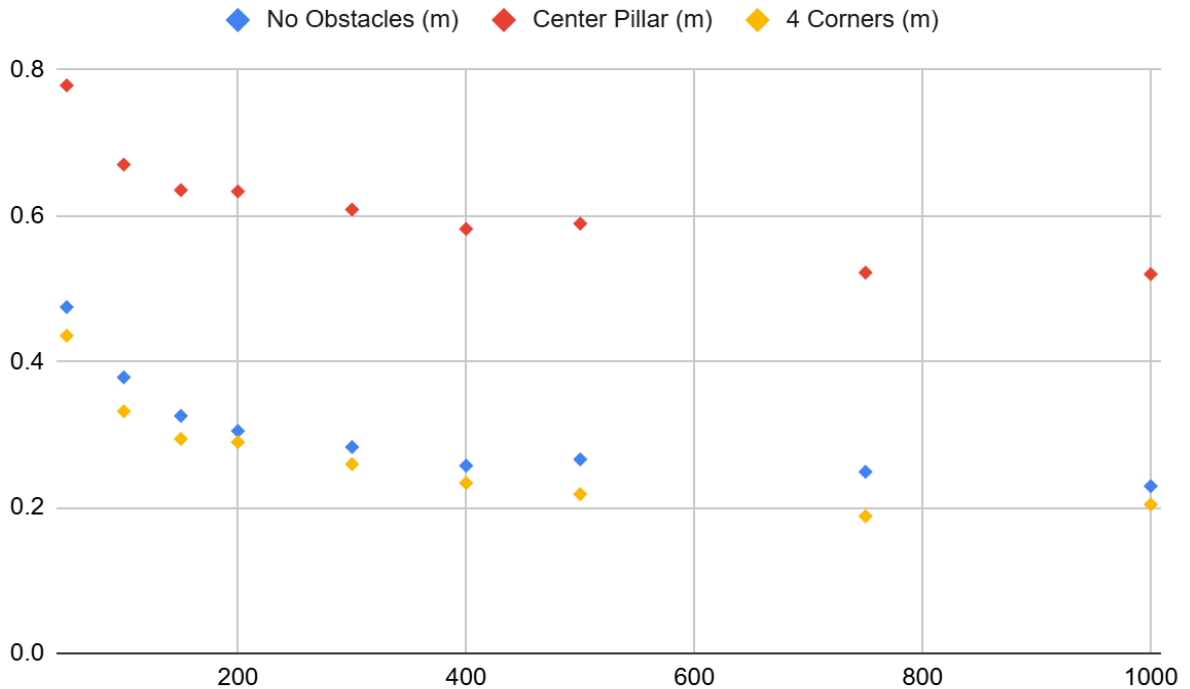


Figure 2: Scatter Plot Diagram of the Mean Localization Error across 1000 Trials in Relation to N Number of Particles Under Different Environments.

6.1 Defining the Curve

To find the equation which clearly demonstrates the relationship between these points to a clear graph, I need to analyze how error changes as the number of samples increases.

let $\bar{X}$ = final estimation,

μ = true position,

N = number of particles used

Then

$$\bar{X} = \frac{\sum_{j=1}^N X_j}{N}$$

To find the variance across all trials

$$Var(\bar{X}) = \frac{\sigma^2}{N}$$

Taking the square root of this results in the standard deviation of the average estimate, equating to the standard deviation of the specific trial itself over the square root of N

$$\sigma_{average} = \frac{\sigma_{individual}}{\sqrt{N}}$$

In this case, the standard deviation measures the typical deviation of the estimate from the true value, and is therefore proportional to the error

$$\sigma_{average} \propto E(N)$$

Therefore, since the $\sigma_{individual}$ is a predetermined constant, the curve is defined with a relationship of

$$E(N) = \frac{k}{\sqrt{N}} + c$$

where k and c are constants and N is the number of trials, the k value controls the rate of decrease while the c constant represents the minimum achievable error.

6.2 Explanation and Regression

An inverse square root relationship makes estimation with limitations nearly impossible to get 100% certainty. There will always be a limit, the constant c , that the algorithm cannot surpass in the case of static estimations. As the number of particles increases, the reduction in error occurs at a diminishing rate. Although the number of particles may reduce the amount of random error, it does not reduce the ambiguity in the non-unique cases where the estimation of the model would return something further off from the actual position.

Firstly, to use regression analysis, I need to linearize the data by

$$let\ x = \frac{1}{\sqrt{N}}$$

Therefore the new relationship becomes

$$E(N) = kx + c$$

Which can be used with linear regression

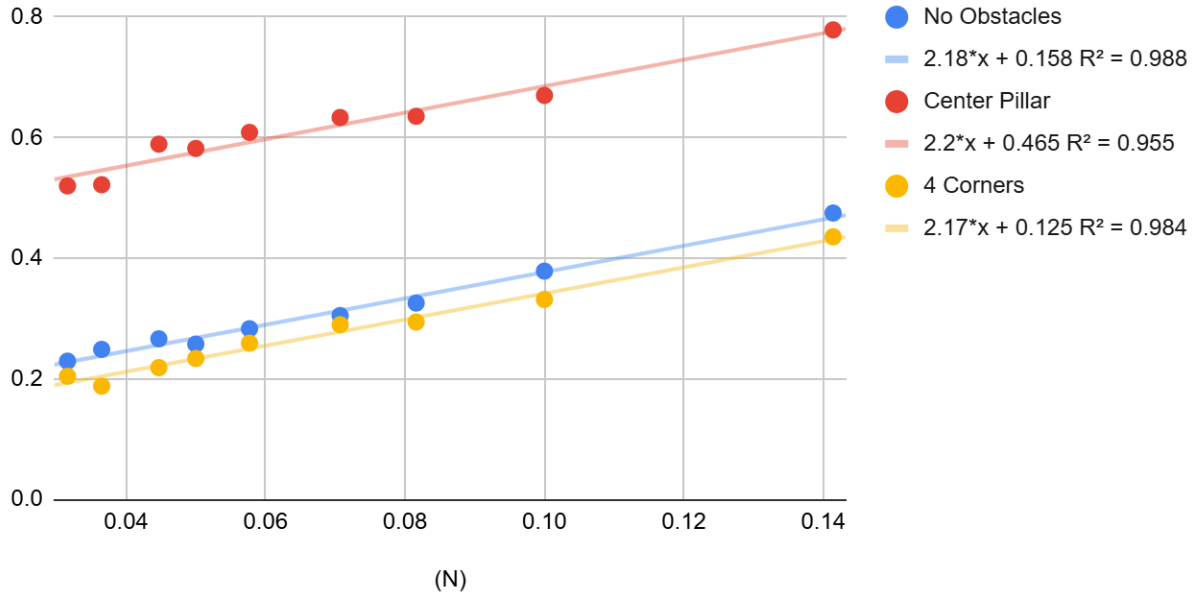


Figure 3: Linearized and Plotted Regression line of a Scatter Plot Diagram of the Mean Localization Error across 1000 Trials in Relation to N Number of Particles Under Different Environments.

The Pearson correlation coefficient suggests that the relationship between my equations and the given data is a strong match, further supporting the idea that the data is best represented using an inverse square root graph. Using the k and c values from the regression line and fitting the values to the old equation, we end with this graph.

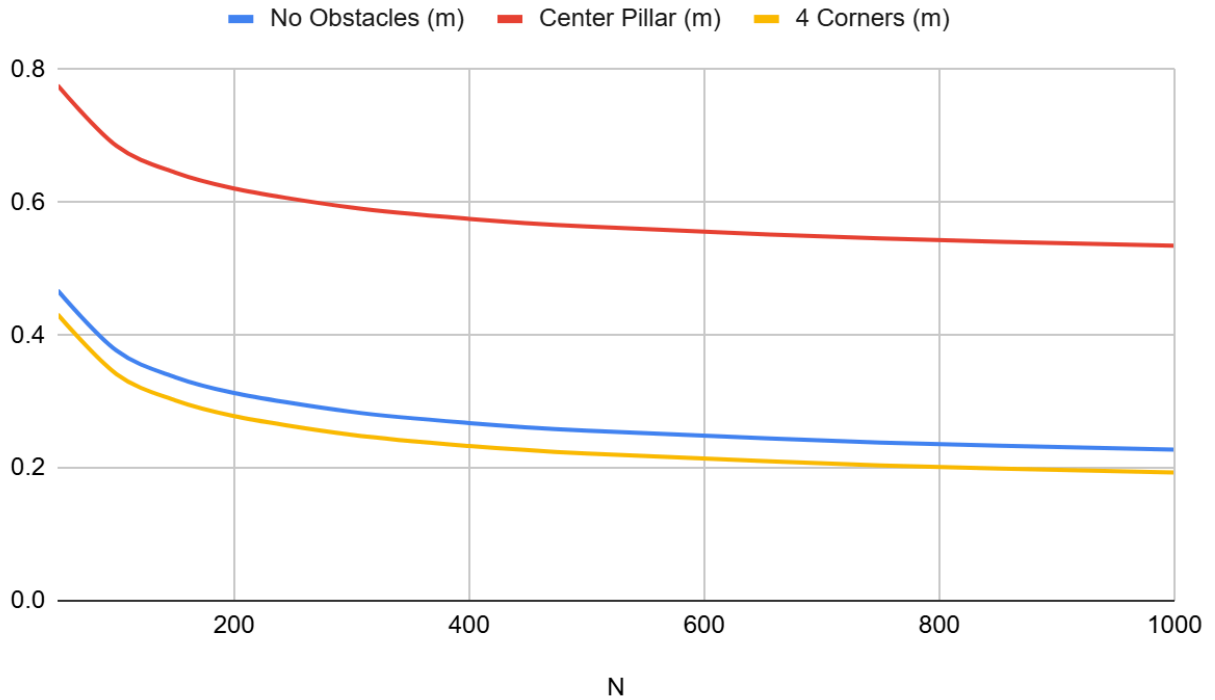


Figure 4: The Equations of the Functions drawn using the Appropriate k and c values.

The values of k across all three graphs remain similar, indicating that the rate in which the increase in particles decreases error is consistent. However, the constant c varies significantly, supporting the idea that environmental structure affects the minimum possible error value rather than affecting the rate of change. The center pillar has the highest minimum possible error, suggesting that it has the most amount of non-unique positions, while the 4 corners map has the least amount of minimum possible error, suggesting the opposite.

7 Environmental Comparison and Reasoning

Taking a look at each individual case, starting with the map with no obstacles the results show that the Monte Carlo algorithm hits a minimum possible error at roughly around 0.158 m

of accuracy. As stated in the limitations in Section 2, there are 4 general situations where non-unique solutions are given, where the robot hits the same side of the wall on both sensors. As a result, particles in completely different locations may receive similar weights, preventing the algorithm from converging before hitting a limit to the accuracy. In this situation with static estimation attempts, the constant c remains a non-zero value as there is a baseline level of uncertainty caused by these environmental factors. For example, a robot with both sensors pointing towards the right wall could produce the exact same sensor readings even if it were placed higher or lower vertically, giving non-unique solutions.

Secondly, although quite counterintuitive at first, the center pillar produces the highest constant value, making the minimum possible error at roughly around 0.465 m of accuracy. This is because the non-unique solutions actually increase instead of decreasing. In this case, along with the 4 general situations with the non obstacle case, the pillar in the middle can act as another wall for the both of the sensors to possibly hit, creating an even wider range of non-unique values. For example, if the robot hits the right wall, then the particles will be spread across the wall, as well as a couple similarly weighted particle clusters near the left side of the pillar because the distances match.

Although the program was run in C++, I made a python version (appendix a) to demonstrate a simple GUI interface displaying particles in the case of the center pillar.

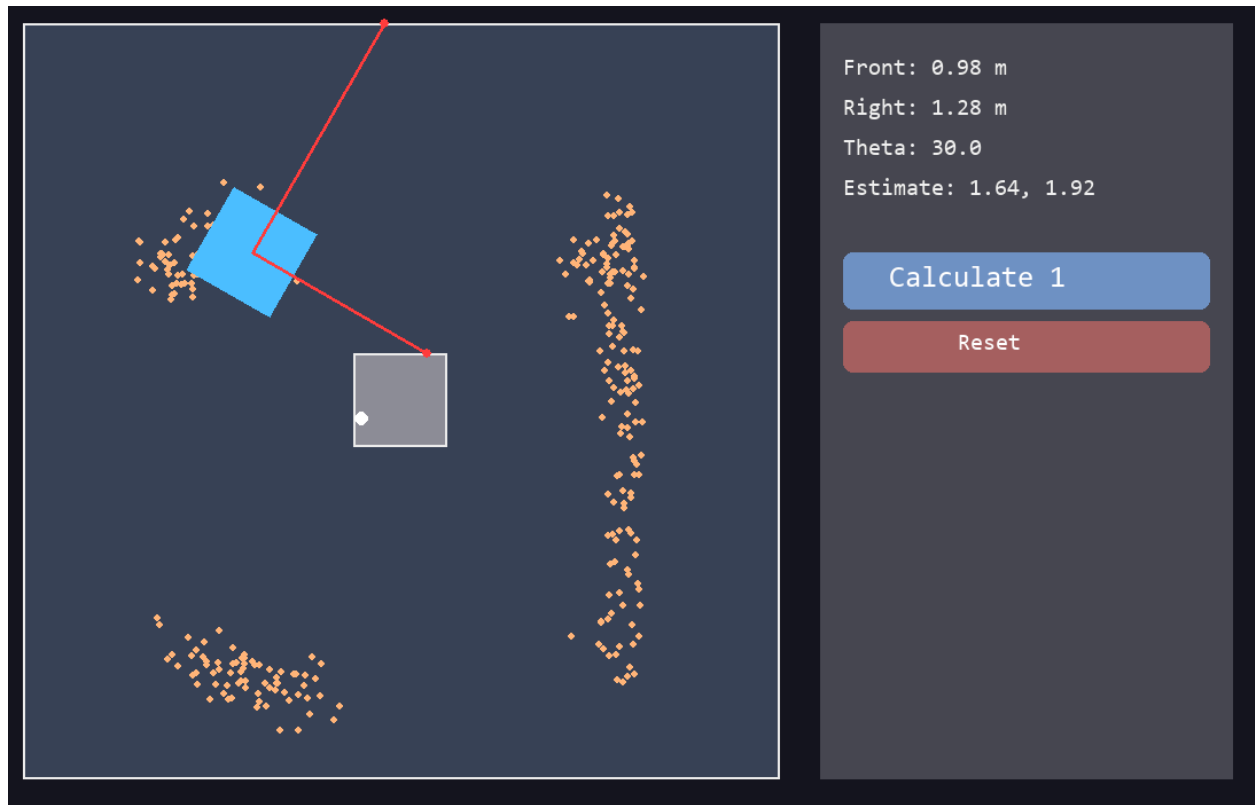


Figure 5: Python Demonstration of the Distribution of 10000 Particles in a Non-Unique Situation

As the demonstration shows, this is another example of a non-unique solution. In this case, the sensor reads 0.98 m in front of it, and 1.28 m to the right of it. However, because of non-unique solutions, the program generates a wide range of particles ranging from the top left, bottom left, and along the right side of the wall because they all produce very similar values to the sensor readings given. As a result, the average of these is taken and ends up being the center of the field, which is inaccurate, contributing to the total error.

Lastly, the 4 corners map produces the lowest constant value, making the minimum possible error at roughly around 0.125 m. Intuitively, this makes sense as the non-unique solutions from before are reduced. For example, if the robot's sensors hit the same wall before, there is less wall overall for the particles to be spread across from because the blocks in the

corners block these particles from being accurate. This causes the average result of each trial to be closer together to the actual robot's position, resulting in a lower error value overall.

Furthermore, since the boxes are so small, it is very difficult for the sensors to hit them at the same time, reducing the overall non-unique solutions that the program can output.

8 Conclusion

The aim of this investigation was to find the relationship between the number of particles N and the localization error within different environments. From both mathematical reasoning and the collected data, it can be concluded that the relationship is strongly correlated with an inverse square root model. As the number of particles increased, the error decreased at a diminishing rate. This is because although the total number of particles decreases the possibility of random variation, it does not eliminate all the limitations that the system itself faces. This is also supported by the data because all the graphs approach a non-zero value.

Although computational limitations of a VEX cortex were not explored in this investigation, through background research and prior knowledge, I concluded that the memory and computational limits for the hardware was within reasonable limits, working comfortably even from values of N up to 50000.

Furthermore, this investigation is incredibly useful in my future career of pursuing engineering. It connects with me on a personal level with my interest in VEX robotics, where determining positions on time is crucial to minimizing error through autonomous runs. Furthermore, localization using estimation is very common in everyday uses, such as self-driving

cars and rovers on different planets which rely on constantly understanding surroundings and positional information.

References

Monte Carlo Localization. *Wikipedia*. Last modified 2026.

https://en.wikipedia.org/wiki/Monte_Carlo_localization.

Particle Filter. *Wikipedia*. Last modified 2026. https://en.wikipedia.org/wiki/Particle_filter.

Appendix

A - Python Simulation Code

```
import math

import random

import pygame

WIDTH = 1280

HEIGHT = 820

M_PER_FOOT = 0.3048

FIELD_SIZE_M = 12.0 * M_PER_FOOT

PIXELS_PER_METER = 55 / M_PER_FOOT

FIELD_W = int(FIELD_SIZE_M * PIXELS_PER_METER)

FIELD_H = int(FIELD_SIZE_M * PIXELS_PER_METER)

FIELD_X = 70

FIELD_Y = 70

PANEL_X = FIELD_X + FIELD_W + 35

PANEL_Y = FIELD_Y

PANEL_W = 360
```

```
PANEL_H = FIELD_H

ROBOT_SIZE_M = 1.5 * M_PER_FOOT

ROBOT_SIZE_PX = ROBOT_SIZE_M * PIXELS_PER_METER

HALF_ROBOT = ROBOT_SIZE_PX / 2

MOVE_SPEED_M = 220.0 * M_PER_FOOT

ROT_SPEED = math.radians(130)

PARTICLE_COUNT = 10000

THETA_SIGMA_DEG = 8.0

SENSOR_SIGMA_M = 0.18 * M_PER_FOOT

PILLAR_SIZE_M = 1.5 * M_PER_FOOT

PILLAR_SIZE_PX = PILLAR_SIZE_M * PIXELS_PER_METER

PILLAR_X = FIELD_X + FIELD_W / 2 - PILLAR_SIZE_PX / 2

PILLAR_Y = FIELD_Y + FIELD_H / 2 - PILLAR_SIZE_PX / 2

PILLAR_RECT = pygame.Rect(int(PILLAR_X), int(PILLAR_Y),
int(PILLAR_SIZE_PX), int(PILLAR_SIZE_PX))

BG = (22, 24, 31)

FIELD_FILL = (58, 68, 88)

FIELD_BORDER = (236, 236, 236)

ROBOT = (78, 190, 255)
```

```
SENSOR = (255, 60, 60)

PILLAR_COLOR = (140, 140, 150)

PANEL_BG = (74, 74, 80)

TEXT = (244, 244, 244)

BUTTON_BG = (112, 145, 195)

BUTTON_HOVER = (132, 166, 218)

RESET_BG = (165, 95, 95)

RESET_HOVER = (190, 115, 115)

def clamp(v, lo, hi):

    return max(lo, min(hi, v))

def wrap_angle(theta):

    while theta <= -math.pi:

        theta += 2 * math.pi

    while theta > math.pi:

        theta -= 2 * math.pi

    return theta

def rotate_point(lx, ly, theta):

    return (
```

```

        lx * math.cos(theta) - ly * math.sin(theta),

        lx * math.sin(theta) + ly * math.cos(theta),

    )

def robot_corners(cx, cy, theta):

    h = ROBOT_SIZE_PX / 2

    pts = [(-h, -h), (h, -h), (h, h), (-h, h)]

    return [(cx + rotate_point(x, y, theta)[0], cy + rotate_point(x, y,
theta)[1]) for x, y in pts]

def point_in_pillar(x, y):

    return PILLAR_RECT.collidepoint(x, y)

def valid_position(x, y):

    if not (FIELD_X <= x <= FIELD_X + FIELD_W and FIELD_Y <= y <= FIELD_Y
+ FIELD_H):

        return False

    if point_in_pillar(x, y):

        return False

    return True

def check_vertical_intersection(x, y, dx, dy, wall_x, y_min, y_max,
best_t):

    if abs(dx) < 1e-12:

```

```

        return best_t

    t = (wall_x - x) / dx

    if t <= 0:

        return best_t

    yy = y + t * dy

    if y_min <= yy <= y_max:

        return min(best_t, t)

    return best_t

def check_horizontal_intersection(x, y, dx, dy, wall_y, x_min, x_max,
best_t):

    if abs(dy) < 1e-12:

        return best_t

    t = (wall_y - y) / dy

    if t <= 0:

        return best_t

    xx = x + t * dx

    if x_min <= xx <= x_max:

        return min(best_t, t)

    return best_t

def add_rect_to_raycast(x, y, dx, dy, rect, best_t):

    best_t = check_vertical_intersection(x, y, dx, dy, rect.left,
rect.top, rect.bottom, best_t)

```

```
    best_t = check_vertical_intersection(x, y, dx, dy, rect.right,
rect.top, rect.bottom, best_t)

    best_t = check_horizontal_intersection(x, y, dx, dy, rect.top,
rect.left, rect.right, best_t)

    best_t = check_horizontal_intersection(x, y, dx, dy, rect.bottom,
rect.left, rect.right, best_t)

    return best_t
```

```
def cast_ray(x, y, angle):
```

```
    dx = math.cos(angle)
```

```
    dy = math.sin(angle)
```

```
    best_t = float("inf")
```

```
    field_rect = pygame.Rect(FIELD_X, FIELD_Y, FIELD_W, FIELD_H)
```

```
    best_t = add_rect_to_raycast(x, y, dx, dy, field_rect, best_t)
```

```
    best_t = add_rect_to_raycast(x, y, dx, dy, PILLAR_RECT, best_t)
```

```
    return best_t
```

```
def sensor(x, y, theta):
```

```
    front = cast_ray(x, y, theta)
```

```
    right = cast_ray(x, y, theta - math.pi / 2)
```

```
    return front / PIXELS_PER_METER, right / PIXELS_PER_METER
```

```
def init_particles(theta):
```

```
    particles = []
```

```

sigma = math.radians(THETA_SIGMA_DEG)

while len(particles) < PARTICLE_COUNT:

    px = random.uniform(FIELD_X, FIELD_X + FIELD_W)

    py = random.uniform(FIELD_Y, FIELD_Y + FIELD_H)

    if not valid_position(px, py):

        continue

    particles.append({

        "x": px,

        "y": py,

        "theta": wrap_angle(random.gauss(theta, sigma)),

        "w": 1 / PARTICLE_COUNT

    })

return particles

def weight_particles(particles, f, r):

    total = 0

    for p in particles:

        pf, pr = sensor(p["x"], p["y"], p["theta"])

        err = (f - pf) ** 2 + (r - pr) ** 2

        p["w"] = math.exp(-err / (2 * SENSOR_SIGMA_M ** 2))

        total += p["w"]

    for p in particles:

        p["w"] /= total if total else 1

```

```
def resample(particles):  
  
    new = []  
  
    cum = []  
  
    s = 0  
  
    for p in particles:  
  
        s += p["w"]  
  
        cum.append(s)  
  
    for _ in particles:  
  
        rr = random.random()  
  
        for i, c in enumerate(cum):  
  
            if rr <= c:  
  
                new.append(**particles[i], "w": 1 / len(particles))  
  
                break  
  
    return new  
  
def estimate(particles):  
  
    if not particles:  
  
        return None  
  
    x = sum(p["x"] for p in particles) / len(particles)  
  
    y = sum(p["y"] for p in particles) / len(particles)  
  
    return x, y
```

```
pygame.init()

screen = pygame.display.set_mode((WIDTH, HEIGHT))

clock = pygame.time.Clock()

font = pygame.font.SysFont("consolas", 20)

big = pygame.font.SysFont("consolas", 26)

x, y = FIELD_X + 200, FIELD_Y + 200

theta = math.radians(30)

particles = []

step = 1

est = None

calc_btn = pygame.Rect(PANEL_X + 20, PANEL_Y + 200, PANEL_W - 40, 50)

reset_btn = pygame.Rect(PANEL_X + 20, PANEL_Y + 260, PANEL_W - 40, 45)

running = True

while running:

    dt = clock.tick(60) / 1000

    mouse = pygame.mouse.get_pos()

    for e in pygame.event.get():
```

```
    if e.type == pygame.QUIT:

        running = False

    if e.type == pygame.MOUSEBUTTONDOWN:

        if calc_btn.collidepoint(e.pos):

            f, r = sensor(x, y, theta)

            if step == 1:

                particles = init_particles(theta)

                est = estimate(particles)

                step = 2

            else:

                weight_particles(particles, f, r)

                particles = resample(particles)

                est = estimate(particles)

                step = 1

        if reset_btn.collidepoint(e.pos):

            particles = []

            est = None

            step = 1

    keys = pygame.key.get_pressed()

    new_x, new_y = x, y

    if keys[pygame.K_w]:
```

```

        new_x += math.cos(theta) * MOVE_SPEED_M * dt * PIXELS_PER_METER

        new_y += math.sin(theta) * MOVE_SPEED_M * dt * PIXELS_PER_METER

    if keys[pygame.K_s]:

        new_x -= math.cos(theta) * MOVE_SPEED_M * dt * PIXELS_PER_METER

        new_y -= math.sin(theta) * MOVE_SPEED_M * dt * PIXELS_PER_METER

    if keys[pygame.K_a]:

        theta -= 2 * dt

    if keys[pygame.K_d]:

        theta += 2 * dt

    theta = wrap_angle(theta)

    new_x = clamp(new_x, FIELD_X, FIELD_X + FIELD_W)

    new_y = clamp(new_y, FIELD_Y, FIELD_Y + FIELD_H)

    if valid_position(new_x, new_y):

        x, y = new_x, new_y

    f, r = sensor(x, y, theta)

    screen.fill(BG)

    pygame.draw.rect(screen, FIELD_FILL, (FIELD_X, FIELD_Y, FIELD_W,
FIELD_H))

    pygame.draw.rect(screen, FIELD_BORDER, (FIELD_X, FIELD_Y, FIELD_W,
FIELD_H), 2)

```

```

pygame.draw.rect(screen, PILLAR_COLOR, PILLAR_RECT)

pygame.draw.rect(screen, FIELD_BORDER, PILLAR_RECT, 2)


for p in particles:

    pygame.draw.circle(screen, (255, 180, 120), (int(p["x"]),
int(p["y"])), 3)


    if est:

        pygame.draw.circle(screen, (255, 255, 255), (int(est[0]),
int(est[1])), 6)


pygame.draw.polygon(screen, ROBOT, robot_corners(x, y, theta))


front_px = f * PIXELS_PER_METER

right_px = r * PIXELS_PER_METER


fx = x + math.cos(theta) * front_px

fy = y + math.sin(theta) * front_px

pygame.draw.line(screen, SENSOR, (x, y), (fx, fy), 3)

pygame.draw.circle(screen, SENSOR, (int(fx), int(fy)), 4)


rx = x + math.cos(theta - math.pi / 2) * right_px

ry = y + math.sin(theta - math.pi / 2) * right_px

```

```

pygame.draw.line(screen, SENSOR, (x, y), (rx, ry), 3)

pygame.draw.circle(screen, SENSOR, (int(rx), int(ry)), 4)


pygame.draw.rect(screen, PANEL_BG, (PANEL_X, PANEL_Y, PANEL_W,
PANEL_H))


if est:

    est_text = f"{{(est[0] - FIELD_X) / PIXELS_PER_METER:.2f}}, {{(est[1]
- FIELD_Y) / PIXELS_PER_METER:.2f}}"

else:

    est_text = "N/A"


info = [

    f"Front: {{f:.2f}} m",

    f"Right: {{r:.2f}} m",

    f"Theta: {{math.degrees(theta) % 360:.1f}}",

    f"Estimate: {{est_text}}"

]


for i, t in enumerate(info):

    screen.blit(font.render(t, True, TEXT), (PANEL_X + 20, PANEL_Y +
30 + i * 35))


calc_color = BUTTON_HOVER if calc_btn.collidepoint(mouse) else
BUTTON_BG

```

```

        reset_color = RESET_HOVER if reset_btn.collidepoint(mouse) else
RESET_BG

pygame.draw.rect(screen, calc_color, calc_btn, border_radius=8)

txt = "Calculate 1" if step == 1 else "Calculate 2"

screen.blit(big.render(txt, True, (255, 255, 255)), (calc_btn.x + 40,
calc_btn.y + 10))

pygame.draw.rect(screen, reset_color, reset_btn, border_radius=8)

screen.blit(font.render("Reset", True, (255, 255, 255)), (reset_btn.x
+ 100, reset_btn.y + 10))

pygame.display.flip()

pygame.quit()

```

C++ program of the simulation

```

#include <algorithm>

#include <cmath>

#include <iomanip>

#include <iostream>

#include <random>

```

```
#include <string>

#include <vector>

struct Particle {

    double x;

    double y;

    double theta;

    double w;

};

struct Pose {

    double x;

    double y;

    double theta;

};

constexpr double PI = 3.14159265358979323846;

constexpr double FT_TO_M = 0.3048;

// Field: 12 ft x 12 ft -> meters

constexpr double FIELD_SIZE_M = 12.0 * FT_TO_M;

// Robot: 18 in = 1.5 ft -> meters
```

```
constexpr double ROBOT_SIZE_M = 1.5 * FT_TO_M;

constexpr double HALF_ROBOT = ROBOT_SIZE_M / 2.0;

// Center pillar: 1.5 ft x 1.5 ft -> meters

constexpr double PILLAR_SIZE_M = 1.5 * FT_TO_M;

constexpr double PILLAR_MIN_X = FIELD_SIZE_M / 2.0 - PILLAR_SIZE_M / 2.0;

constexpr double PILLAR_MAX_X = FIELD_SIZE_M / 2.0 + PILLAR_SIZE_M / 2.0;

constexpr double PILLAR_MIN_Y = FIELD_SIZE_M / 2.0 - PILLAR_SIZE_M / 2.0;

constexpr double PILLAR_MAX_Y = FIELD_SIZE_M / 2.0 + PILLAR_SIZE_M / 2.0;


// Four corner boxes: total area = robot area

// Original side length was 0.75 ft -> meters

constexpr double CORNER_BOX_SIZE_M = 0.75 * FT_TO_M;


// MCL parameters

constexpr double THETA_SIGMA_DEG = 8.0;

constexpr double SENSOR_SIGMA_M = 0.18 * FT_TO_M;


enum EnvironmentType {

    EMPTY_FIELD,

    CENTER_PILLAR,

    CORNER_BOXES

};
```

```

double wrap_angle(double theta) {

    while (theta <= -PI) theta += 2.0 * PI;

    while (theta > PI) theta -= 2.0 * PI;

    return theta;

}

bool point_in_rect(double x, double y, double min_x, double max_x, double
min_y, double max_y) {

    return (min_x <= x && x <= max_x && min_y <= y && y <= max_y);

}

bool inside_pillar(double x, double y) {

    return point_in_rect(x, y, PILLAR_MIN_X, PILLAR_MAX_X, PILLAR_MIN_Y,
PILLAR_MAX_Y);

}

bool inside_corner_box(double x, double y) {

    // bottom-left

    if (point_in_rect(x, y, 0.0, CORNER_BOX_SIZE_M, 0.0,
CORNER_BOX_SIZE_M)) return true;

    // bottom-right

    if (point_in_rect(x, y, FIELD_SIZE_M - CORNER_BOX_SIZE_M,
FIELD_SIZE_M, 0.0, CORNER_BOX_SIZE_M)) return true;

    // top-left

```

```

        if (point_in_rect(x, y, 0.0, CORNER_BOX_SIZE_M, FIELD_SIZE_M -
CORNER_BOX_SIZE_M, FIELD_SIZE_M)) return true;

        // top-right

        if (point_in_rect(x, y, FIELD_SIZE_M - CORNER_BOX_SIZE_M,
FIELD_SIZE_M,

                                FIELD_SIZE_M - CORNER_BOX_SIZE_M, FIELD_SIZE_M))
return true;

        return false;
    }

bool valid_robot_position(double x, double y, EnvironmentType env) {

    if (x < HALF_ROBOT || x > FIELD_SIZE_M - HALF_ROBOT ||

        y < HALF_ROBOT || y > FIELD_SIZE_M - HALF_ROBOT) {

        return false;

    }

    if (env == CENTER_PILLAR && inside_pillar(x, y)) {

        return false;

    }

    if (env == CORNER_BOXES && inside_corner_box(x, y)) {

        return false;

    }

```

```

        return true;
    }

void check_vertical_intersection(double x, double y, double dx, double dy,
                                double wall_x, double ymin, double ymax,
                                double& best_t) {

    if (std::abs(dx) < 1e-12) return;

    double t = (wall_x - x) / dx;

    if (t <= 0.0) return;

    double yy = y + t * dy;

    if (ymin <= yy && yy <= ymax) {

        best_t = std::min(best_t, t);

    }
}

void check_horizontal_intersection(double x, double y, double dx, double
dy,
                                double wall_y, double xmin, double
xmax,
                                double& best_t) {

    if (std::abs(dy) < 1e-12) return;

    double t = (wall_y - y) / dy;

    if (t <= 0.0) return;

```

```

        double xx = x + t * dx;

        if (xmin <= xx && xx <= xmax) {

            best_t = std::min(best_t, t);

        }
    }

}

void add_rect_to_ray_cast(double x, double y, double dx, double dy,

                        double min_x, double max_x, double min_y, double

max_y,

                        double& best_t) {

    check_vertical_intersection(x, y, dx, dy, min_x, min_y, max_y,

best_t);

    check_vertical_intersection(x, y, dx, dy, max_x, min_y, max_y,

best_t);

    check_horizontal_intersection(x, y, dx, dy, min_y, min_x, max_x,

best_t);

    check_horizontal_intersection(x, y, dx, dy, max_y, min_x, max_x,

best_t);

}

double cast_ray(double x, double y, double angle, EnvironmentType env) {

    double dx = std::cos(angle);

    double dy = std::sin(angle);

    double best_t = 1e18;

```

```
// Outer field walls

add_rect_to_ray_cast(x, y, dx, dy, 0.0, FIELD_SIZE_M, 0.0,
FIELD_SIZE_M, best_t);

if (env == CENTER_PILLAR) {

    add_rect_to_ray_cast(x, y, dx, dy,

                        PILLAR_MIN_X, PILLAR_MAX_X,

                        PILLAR_MIN_Y, PILLAR_MAX_Y,

                        best_t);

}

if (env == CORNER_BOXES) {

    // bottom-left

    add_rect_to_ray_cast(x, y, dx, dy,

                        0.0, CORNER_BOX_SIZE_M,

                        0.0, CORNER_BOX_SIZE_M,

                        best_t);

    // bottom-right

    add_rect_to_ray_cast(x, y, dx, dy,

                        FIELD_SIZE_M - CORNER_BOX_SIZE_M,

FIELD_SIZE_M,

                        0.0, CORNER_BOX_SIZE_M,

                        best_t);
```

```

        // top-left

        add_rect_to_ray_cast(x, y, dx, dy,

                             0.0, CORNER_BOX_SIZE_M,

                             FIELD_SIZE_M - CORNER_BOX_SIZE_M,
FIELD_SIZE_M,

                             best_t);

        // top-right

        add_rect_to_ray_cast(x, y, dx, dy,

                             FIELD_SIZE_M - CORNER_BOX_SIZE_M,
FIELD_SIZE_M,

                             FIELD_SIZE_M - CORNER_BOX_SIZE_M,
FIELD_SIZE_M,

                             best_t);

    }

    return best_t;
}

std::pair<double, double> sensor_readings(double x, double y, double
theta, EnvironmentType env) {

    double front = cast_ray(x, y, theta, env);

    double right = cast_ray(x, y, theta - PI / 2.0, env);

    return {front, right};
}

```

```

}

std::vector<Particle> initialize_particles(int N, double measured_theta,
EnvironmentType env, std::mt19937& rng) {

    std::uniform_real_distribution<double> pos_dist(HALF_ROBOT,
FIELD_SIZE_M - HALF_ROBOT);

    std::normal_distribution<double> theta_dist(measured_theta,
THETA_SIGMA_DEG * PI / 180.0);

    std::vector<Particle> particles;

    particles.reserve(N);

    while ((int)particles.size() < N) {

        double x = pos_dist(rng);

        double y = pos_dist(rng);

        if (!valid_robot_position(x, y, env)) continue;

        Particle p;

        p.x = x;

        p.y = y;

        p.theta = wrap_angle(theta_dist(rng));

        p.w = 1.0 / N;

        particles.push_back(p);
    }
}

```

```

    }

    return particles;
}

void compute_weights(std::vector<Particle>& particles, double true_front,
double true_right, EnvironmentType env) {

    double total = 0.0;

    for (auto& p : particles) {

        auto [pred_front, pred_right] = sensor_readings(p.x, p.y, p.theta,
env);

        double err_f = true_front - pred_front;

        double err_r = true_right - pred_right;

        double wf = std::exp(-(err_f * err_f) / (2.0 * SENSOR_SIGMA_M *
SENSOR_SIGMA_M));

        double wr = std::exp(-(err_r * err_r) / (2.0 * SENSOR_SIGMA_M *
SENSOR_SIGMA_M));

        p.w = wf * wr;

        total += p.w;

    }
}

```

```

    if (total <= 1e-15) {

        double uniform = 1.0 / particles.size();

        for (auto& p : particles) p.w = uniform;

        return;

    }

    for (auto& p : particles) {

        p.w /= total;

    }
}

std::vector<Particle> resample_particles(const std::vector<Particle>&
particles, std::mt19937& rng) {

    int N = (int)particles.size();

    std::vector<double> cumulative;

    cumulative.reserve(N);

    double sum = 0.0;

    for (const auto& p : particles) {

        sum += p.w;

        cumulative.push_back(sum);

    }
}

```

```

std::uniform_real_distribution<double> u(0.0, 1.0);

std::vector<Particle> new_particles;

new_particles.reserve(N);

for (int i = 0; i < N; ++i) {

    double r = u(rng);

    auto it = std::lower_bound(cumulative.begin(), cumulative.end(),
r);

    int idx = (int)(it - cumulative.begin());

    Particle chosen = particles[idx];

    chosen.w = 1.0 / N;

    new_particles.push_back(chosen);

}

return new_particles;
}

Pose estimate_pose(const std::vector<Particle>& particles) {

    Pose est{};

    est.x = 0.0;

    est.y = 0.0;

```

```

double sin_sum = 0.0;

double cos_sum = 0.0;

for (const auto& p : particles) {

    est.x += p.x;

    est.y += p.y;

    sin_sum += std::sin(p.theta);

    cos_sum += std::cos(p.theta);

}

est.x /= particles.size();

est.y /= particles.size();

est.theta = std::atan2(sin_sum, cos_sum);

return est;
}

double positional_error(const Pose& truth, const Pose& estimate) {

    double dx = truth.x - estimate.x;

    double dy = truth.y - estimate.y;

    return std::sqrt(dx * dx + dy * dy);

}

```

```
double run_experiment_for_N(int N, int TRIALS, EnvironmentType env,
std::mt19937& rng) {

    std::uniform_real_distribution<double> pos_dist(HALF_ROBOT,
FIELD_SIZE_M - HALF_ROBOT);

    std::uniform_real_distribution<double> theta_dist(-PI, PI);

    double total_error = 0.0;

    for (int trial = 0; trial < TRIALS; ++trial) {

        Pose truth;

        do {

            truth.x = pos_dist(rng);

            truth.y = pos_dist(rng);

            truth.theta = theta_dist(rng);

        } while (!valid_robot_position(truth.x, truth.y, env));

        auto [front_true, right_true] = sensor_readings(truth.x, truth.y,
truth.theta, env);

        std::vector<Particle> particles = initialize_particles(N,
truth.theta, env, rng);

        compute_weights(particles, front_true, right_true, env);

        particles = resample_particles(particles, rng);
```

```

        Pose est = estimate_pose(particles);

        total_error += positional_error(truth, est);

    }

    return total_error / TRIALS;
}

void run_environment(const std::string& name, EnvironmentType env, const
std::vector<int>& particle_counts, int trials, std::mt19937& rng) {

    std::cout << "=== " << name << " ===\n";

    std::cout << "N,AverageError_m\n";

    for (int N : particle_counts) {

        double avg_error = run_experiment_for_N(N, trials, env, rng);

        std::cout << N << "," << avg_error << "\n";

    }

    std::cout << "\n";
}

int main() {

    std::random_device rd;

    std::mt19937 rng(rd());

    std::vector<int> particle_counts = {50, 100, 150, 200, 300, 400, 500,
750, 1000};

```

```
const int TRIALS = 1000;

std::cout << std::fixed << std::setprecision(4);

run_environment("EMPTY FIELD", EMPTY_FIELD, particle_counts, TRIALS,
rng);

run_environment("FIELD WITH CENTER PILLAR", CENTER_PILLAR,
particle_counts, TRIALS, rng);

run_environment("FIELD WITH FOUR CORNER BOXES", CORNER_BOXES,
particle_counts, TRIALS, rng);

return 0;
}
```